

Wergonic — Bug Report

Everything I found wrong in the `dev` release.

Tested on: dev.cloud.wergonic.com (not production) · organisation *Test Hub* · 13 July 2026

Data: I created, changed and deleted nothing. Every delete button was clicked as far as the confirmation box, then cancelled.

■ **Serious** — wrong results, or data shown that isn't true. ■ **Important** — works, but wrong or misleading. ■ **Minor**

1 MEC calculates the wrong things

Where Any generated MEC assessment.

Problem Four separate faults:

- Questions **12.1 and 12.2** are never calculated. They come back empty on every MEC ever produced.
- Questions **11.1 and 11.2** ("head in red zone") measure the difference between head and trunk instead of the head angle. So question 11 is quietly doing question 12's job — and a worker whose head moves with their body always scores near zero.
- A **unit mistake**: a value in minutes is saved as if it were seconds, then divided by 60 again. The result is 60× too small, which makes the app's own risk levels (4 / 8 / 15 minutes per hour) impossible to reach.
- Head and trunk readings are matched **by position in the list, not by time**. The two sensors record at different rates, so it compares moments that didn't happen together.

Seen: generated MECs 2177 and 2178 return *every single number as zero*, with questions 12.1 and 12.2 empty.

CODE EVIDENCE

- **12.1 / 12.2 never calculated** — `ramp/calculations/mec_calculator.py` declares both keys in the response dictionary and **never assigns them a value**. They go out as `null` every time.
- **11.1 / 11.2 measure the wrong thing** — the "head in red zone" calculation subtracts the trunk angle from the head angle. That difference is what question **12** was defined to measure. Question 11 was supposed to use the head angle on its own.
- **The unit mistake** — the helper that measures static time returns **minutes**. Its result is written into a field called `static_seconds`, and later divided by 60 to produce `static_minutes_per_hour`. So it's divided by 60 once too often — the number is **60× too small**, and the app's 4 / 8 / 15 min-per-hour risk levels can never be reached.
- **Matching by position, not time** — the head and trunk readings are paired by their index in the list. The two sensors don't produce the same number of readings, so from the first dropped sample onwards it is comparing the head at one moment with the trunk at another.

2 A generated RAMP looks finished when it isn't

Where Risk assessments → any generated RAMP → results page.

Problem The backend marks it "Completed" as soon as the *calculation* finishes. The app reads "Completed" as *the assessment is done*. So a RAMP with only the postures filled in shows a green **Completed** chip, a **Download PDF** button, and a risk bar reading **0 red / 0 yellow / 5 green**.

Seen: assessment 2224 — 6 posture questions answered, roughly 34 never touched, presented as complete and green.

CODE EVIDENCE

- `ramp/calculations/ramp_calculator.py:497` — the status is set to `COMPLETED` at the *end of the calculation*. The developer's own comment above it says so: "*Generation finished → COMPLETED... IN_PROGRESS now only means 'still calculating'.*"
- `features/RAMP/.../RAMPResultsV2.tsx:370` — the frontend works out whether the manual part is done with `manualDone = assessment.source === MANUAL`. So **source is doing two jobs at once**: "where did this come from" and "has a human finished it". That's why the Generated tag disappears when someone completes the RAMP.

Fix Add a **Generated** status between "In progress" and "Completed". Make the risk bar say "5 of 40 answered" instead of "5 green". Don't offer the PDF until it's really complete, or stamp it "Partial". Add a way to reopen a completed assessment — today there is none.

Once the status exists, **drop the "Generated" tag from the list**. It's redundant, and it's unreliable for the reason above. The measurement number already shown in the row (`#7010`) does the job better and never disappears.

3 Drop one header and the API serves another customer's data

Where Every screen that shows organisation data.

Problem Web and mobile each keep their own selected organisation, and a header tells the API which one to use. That part is correct — web never overwrites mobile's choice.

But if a user has never touched the web organisation switcher, **the web falls back to whatever organisation their phone is on**. And when a user belongs to several, that fallback picks one **at random**.

Seen: same account, same URL — one request with the header, one without. One returned **20 assessments from Test Hub**. The other returned **11 completely different assessments from another organisation**. No error, no warning.

CODE EVIDENCE

- `users/scoping.py:24` — the whole thing hinges on one header:

```
if request.headers.get("X-Client-Platform","").lower() == "web": org = user.get_active_consulting_organization()
else: org = user.get_active_organization() ← the mobile organisation.
```

So **any request that forgets the header silently gets the mobile organisation instead of the web one**. It doesn't fail — it just returns the wrong data.
- `users/models/baseUserModel.py:199-208` — `get_active_consulting_organization()` ends with `"Fallback to active mobile organization" → return self.get_active_organization()`. So if the user has never used the web switcher, **the web follows the phone**.
- `users/models/baseUserModel.py:189-192` — that mobile lookup is `UserOrganization.objects.filter(user=self, is_active=True).first()`. **There is no ordering**. With more than one row marked active, `.first()` returns whichever the database happens to hand back.
- And more than one *is* marked active: your own account has **three** (Test Hub, VolvoCars, Wergonic). Your login token also carries `"organization": null` — meaning **you personally are in the fallback path right now**.

Fix Set the web organisation properly on first web login instead of falling back to the phone's. Make the mobile lookup deterministic (only one row should ever be active — clean up the duplicates). And if the header is missing, reject the request instead of guessing.

4 Six checkboxes in "Generate report" do nothing

Where Measurements → Generate report.

Problem Of 17 checkboxes, 11 work and **6 have no effect at all** — ticking or unticking them produces an identical Word file:

- **Worker Details** — the worker's name is always printed
- **Recorder Details** — always printed
- **Session Duration** — always printed
- **Session Metadata** — there is no such section in the report at all
- **AI Risk Summary** — the AI text is printed as part of "Grouped Risk Summary", so it only disappears if you untick *that* box instead
- **Appendix (Raw Data)** — no effect

Seen: generated a real report on #7010 with all four "Session Information" boxes unticked, opened the Word file — the table was still there.

CODE EVIDENCE

- `user_sessions/services/report_service.py:82-98` — this is the function that reads the checkboxes. **worker, recorder, duration and metadata are not in it.** The frontend sends them; the backend never looks at them.
- `report_service.py:57` and `:63` — `appendix` and `risk_summary_text` are read into the options object, and then **never used anywhere**. Two flags that go nowhere.
- `report_service.py:70-73, 83` — the header table is controlled by a key called `session_info`, and the default when the key is absent is **include it** (`_parse_bool(None, default=True)`). **The frontend never sends session_info**. So it's always on, and no checkbox can turn it off.
- `utils/session_docx_report.py:42-43` — the AI summary paragraph is written from inside the *grouped risk summary* function. The purpose-built function for it (`:770`) exists but is **never called by anything**.

Fix Delete all six. The four "Session Information" ones pretend to control the header table (Session ID, dates, duration, workplace) — and that table is mandatory, the report is meaningless without it. A section that must always be there shouldn't be offered as removable.

5 The same numbers don't match between RAMP and the measurement screen

Where Generated RAMP vs measurement detail vs Word report.

- Problem The same maths is written **three separate times**, in three separate files — once for the RAMP, once for the measurement screen, once for the report. They don't agree:
- The RAMP **chops the decimals off angles** before comparing. So its "more than 30°" really means "31° or more". The measurement screen counts anything above 30.0°.
 - The RAMP measures percentage **of time**. The measurement screen measures percentage **of samples**. Two different percentages from the same data, even when the threshold is identical.
 - A typo in the arm calculation means a filter that should run **never runs**, so the RAMP's arm percentage is worked out over a different total than the chart's.
 - For trunk side bending, item 1.3 needs "30 or below" and item 1.4 needs "above 30". A bend of **exactly 30° is counted by neither** — but the measurement chart does count it.
 - The Word report uses a **third set of thresholds again**, so the report, the RAMP and the measurement screen can each show a different number for the same thing.

CODE EVIDENCE

- **The three copies:** the RAMP reads `utils/mec_mapping.py:317` · the measurement screen reads `calculations/pie_stats.py:29` · the Word report reads `calculations/risk_assessment_posture.py:36` . Three files, same maths, written three times.
- **The decimal chopping:** `mec_mapping.py:335` does `v = int(v)` before comparing, and `:347` uses a strict `>` . So a 30.7° bend becomes 30, and "more than 30" rejects it. `pie_stats.py:180` compares the real number.
- **Time vs samples:** `mec_mapping.py:357-359` divides milliseconds by milliseconds. `pie_stats.py:180` uses `.mean()` — a count of readings. These only give the same answer if the sensor's timing is perfectly even, which it isn't.
- **The arm typo:** `mec_mapping.py:324` checks for "Elevation_angle" (capital E) but `ramp_calculator.py:399-400` passes "elevation_angle" (lowercase). **The filter never fires.** The chart drops negative readings; the RAMP keeps them.
- **The 30° gap:** `ramp_calculator.py:313` uses `10 < v <= 29` , and `:343` uses `v > 30` . Nothing catches exactly 30. `pie_stats.py:638-640` uses `10 < v <= 30` , so the chart does.
- **The third threshold set:** `risk_assessment_posture.py:162` uses `-10°` for backward bending where the RAMP uses `-5°`. Its own comment reads "Use -10° (not -5°) consistently" — **someone already fixed this, but only in the report file.**

Fix Put this maths in **one** file that all three read from. As long as it's written three times it will keep drifting.

6 Clicking "Generate" twice creates two identical assessments

Where Any measurement.

Problem No check for an existing assessment, and no protection against a double click.

Seen: measurement #7010 has **five** RAMP assessments. Two of them are byte-for-byte identical.

Note Several assessments per measurement is normal — that's not the issue. Identical duplicates from one button is.

7 Assessments get stuck at "In progress" forever

Where Risk assessments list.

Problem If the background worker dies mid-generation, nothing ever touches the row again. No timeout, no retry, no cleanup. The row also *looks* clickable (the cursor changes) but clicking does nothing — the only thing you can do is delete it.

Seen: nine assessments in Test Hub stuck at "In progress" with **nothing inside them**.

8 Active measurements and the rest of the app disagree

Where Home vs Measurements → Active tab.

Problem The Active tab reads from a different database (Firestore) than everything else (Postgres). They don't match.

Seen: Home says "**Failed / interrupted: 0**". On the same refresh, the Active tab lists **six interrupted measurements**.

Those sessions are stuck. #6842 shows a duration of **1,429 hours** — about 59 days. It started on 14 May and the counter was *still going up* between two screenshots.

Worse The Actions column was removed from that tab in a recent commit, so **there is now no way to end a stuck session from the app at all**.

9 Every count on Home stops at 10

Where Home → the Inbox badges and the "N things need your attention" headline.

Problem The app asks the server for 10 items, then counts how many came back. The number can therefore never go above 10, no matter how big the real backlog is.

Seen: the server reports **185** finished measurements. Home looks at the newest 10, finds 6 without an assessment, and states "**6**".

Worse The "Run risk assessment" count fetches the 10 newest measurements and *then* checks which have no assessment. If those 10 all have one, it shows **0** — even with 200 older measurements waiting.

CODE EVIDENCE

- `features/home/hooks/useHomeInbox.ts` — all three queries are hardcoded to `page_size: "10"`.
- `features/home/.../Inbox.tsx:39` — the badge is `items.filter(...).length`. It counts **what came back**, not what the server said exists. The server's real total is in the `count` field of the same reply, and it's ignored.
- `pages/dashboard/HomePage/NewHomePage.tsx:57` — the summary tile **on the very same page** does read the real `count`. So Home shows two different numbers for the same set of records.
- `pages/dashboard/HomePage/newHomeMappers.tsx:133` — "needs an assessment" is worked out in the browser with `(s.assessments ?? []).length === 0`, *after* only 10 rows were fetched. The server is never asked the question.

10 Merging a condition always fails with a server error

Where Work Library → merge a condition in a single-choice group.

Problem The code adds the new links before removing the old ones, so the "only one choice allowed" rule trips every time. It fails always — **except** when the condition isn't used anywhere, which is exactly when merging is pointless. Single-choice is the default type.

User sees "Something went wrong — please try again."

CODE EVIDENCE

- `work_library/views.py:178-186` — the order is: **create the target's links first, delete the source's links after**. At the moment of creation the source's row still exists, so the model's own "only one choice per group" validation sees two and refuses.
- It raises a Django `ValidationError`, which the API layer doesn't know how to handle → it becomes an **HTTP 500** instead of a readable message. That's why the user only ever sees "Something went wrong".
- Swapping the two steps (delete first, then create) fixes it.

Important

11 RAMP item 1.3 says one threshold and uses another

- Where** Generated RAMP → item 1.3, "Back posture — moderate bending".
- Problem** The screen says "*forward bending more than 30 degrees*". The code measures the **20°–45° band**. Those are different things. It reuses the sentence from item 1.1 — the correct sentence was never written.
- Result** The RAMP screen shows **the same sentence twice** — on items 1.1 and 1.3 — with two different percentages under it. Same problem on the side-bending line.

CODE EVIDENCE

- `ramp_calculator.py:310` — item 1.3 is calculated as `20 < v <= 45`. The result is stored in a field literally named `...forward_bending_20_44_degrees_percentage`. **The code knows it's the 20–45 band.**
- `features/RAMP/hooks/generatedDetailedSummary.tsx:123` — but the screen renders the text key `calculation_forward_bending_30_degrees`, which reads "*...more than 30 degrees*" (`en.json:504`).
- `generatedDetailedSummary.tsx:44` — **that same key is also used for item 1.1**, where it *is* correct. So the sentence gets printed twice on one screen with different numbers under it.
- There is **no** `calculation_forward_bending_20_44_degrees` text key anywhere in the codebase. It was never written — item 1.3 just borrowed 1.1's.

12 A body part that wasn't measured shows as "Acceptable"

- Where** Generated RAMP → item 1.6, "hand in or outside the outer work area".
- Problem** The item has **no data at all** — it says "Missing: Forearm", total time `00:00:00`, no zone. But the card still reads "**Score: 0 — Acceptable**".
- Fix** It should say "Not assessed". A body part you didn't measure must never be shown as green.

13 The Inbox and Recent activity show the same things

Where Home.

Problem Every item in the Inbox also appears in Recent activity. There's even a third feed doing the same job — the notification bell.

And "To review" can never empty, because it's literally "every assessment that finished calculating". It's an archive with a task list's name on it.

Fix

Inbox = things that went wrong or are waiting on you. Recent activity = things that went fine. Nothing in both.

- **Inbox:** failed measurements · disconnected measurements · finished measurements with no assessment yet · **generated assessments waiting for their manual questions** · **assessments that failed to generate** (currently nowhere) · your own unfinished drafts.
- **Recent activity:** a measurement finished · an assessment finished · a worker was added. That's all.
- **Remove from Recent activity:** failed and disconnected measurements, and failed assessments — those are the Inbox's job. Also remove "assessment generated", because "assessment completed" fires seconds later for the same record, so every generated assessment currently appears **twice**.

Backend work: two small filters. One to filter assessments by whether a human answered them, one to filter measurements by whether they have an assessment. Everything else already exists.

CODE EVIDENCE

- `features/home/hooks/useHomeInbox.ts:37-41` — the "To review" list is literally `/assessments/?status=COMPLETED`. Since `COMPLETED` is set by the calculator (see bug #2), this is **"every assessment that ever finished calculating"**. Nothing can ever leave it.
- `activity/signals.py:51-56, 84-89` — the activity feed fires on exactly the same events the Inbox lists: failed, disconnected, completed, assessment-generated, assessment-completed. That's the duplication, in one file.
- `activity/signals.py:84-89` — a generated assessment fires `ASSESSMENT_GENERATED` **when it's created and** `ASSESSMENT_COMPLETED` **seconds later** when the calculation ends. Two rows, one assessment, every time.
- **The fix is already sitting in your data.** `AssessmentFormControls.hooks.ts:109-115` — when a human submits the form, the app sends `{status: COMPLETED, source: MANUAL}`. And `RAMPResultsV2.tsx:370` already reads `source === MANUAL` as "a human finished this". So **"waiting for review" = `status=COMPLETED AND source=generated`**.
It just isn't filterable: `ramp/filters.py:39-43` exposes `status`, `assessment_category`, `created_by_id` — but **not** `source`. Adding `"source": ["exact"]` is one line, and it turns the Inbox into a real queue that empties itself.

14 "Dismissed" in the Inbox only works in one browser

Where Home → Inbox.

Problem Dismissing is stored in the browser, not on the server. Open the app on another computer and it's back. A colleague handling the item doesn't clear it from your list.

Worse: the "Review" button marks an item handled **just for opening it** — look at an assessment, do nothing, and it's gone from your inbox.

Also Neither widget has an empty state — an empty inbox is a blank box. And archived measurements still appear in the Inbox.

CODE EVIDENCE

- `atoms/other.ts:14` — `atomWithStorage("home-inbox-dismissed", [])`. That's browser `localStorage`. It is **not** saved on the server, so it doesn't follow the user and doesn't clear for anyone else.
- `NewHomePage.tsx:74-76` — the "Review" handler adds the item to that dismissed list **at the moment it navigates**. Opening the assessment is enough; you don't have to do anything to it.
- The list is also capped at 200 ids (`NewHomePage.tsx:44-47`), so past that, old dismissals silently come back.

15 "Measured this month" is always 0

Where Workers page.

Problem The app reads the number from the wrong place in the server's reply, so it's always empty and always shows 0 — plus a permanent "+0% vs last month".

Seen: the card says 0, while worker Farhad Abtahi's own page shows **175 measurements, last measured this month**.

CODE EVIDENCE

- `features/workers/hooks/useWorkersStats.ts:37` — the server replies with `{ stats: { sessions_count: ... } }`, but the code reads `sessions_count` straight off the top level. It's one nesting level too shallow, so it's always undefined, and `undefined ?? 0` gives **0**.
- The app's own TypeScript type says the reply is wrapped in `stats` — but the line casts it `as any`, which **switches the compiler's check off**. That's why nobody caught it.
- Fix: `.data.stats` instead of `.data`, and delete the `as any`.

16 Three different worker counts, and admins are counted as workers

Where Workers page, Home, licence bar.

Problem Workers says **23**. Home says **5**. The licence bar says **12**. All three are labelled "workers".

On top of that, **every user account automatically gets a worker profile** and the list never filters them out — so every admin and ergonomist is listed and counted as a worker.

And the "Active workers" number is fed by the search box, so **typing in search changes the count**.

17 Merge leaves ghost measurements and doubles every event

Where Measurements → Merge.

Problem The original measurements aren't deleted. Their data is moved out, but they stay in the list looking like normal completed measurements — **with nothing inside them**.

Events are copied to the merged measurement *and then* moved to it as well, so **every event ends up on it twice**.

Work Library tags (work cycles, activities, conditions) are **not** carried over — the merged measurement loses all its tagging.

18 Notifications name the wrong thing

Where Notification bell, Recent activity.

Problem The bell says "RAMP generated from measurement **#2224**". 2224 is the *assessment* number — the measurement is #7010.

In Recent activity, assessments with no measurement print the literal text "**measurement #None**". And the five different RAMPs on #7010 all print as identical lines, because the text doesn't include the assessment number.

19 Exposure limit colours are wrong, and the head has no limits

Where Measurement detail → Exposure limits.

Problem Colours are assigned by row position, not by risk. So on the Head and Trunk cards the **safest band (0–10°) is yellow** and **backward bending is green**.

The **Head card shows "—" for every recommended limit** — yet its bands are still coloured red and amber. A risk colour with no threshold behind it.

Two cards are both titled "Arm angular velocity", with different numbers.

20 Home shows zeros instead of an error

Where Home.

Problem Home never checks whether its data loaded. If the backend is down it renders a complete, confident dashboard with **every number reading 0** — which looks exactly like a healthy but empty organisation.

Note This is a step backwards — the old Home had a loading skeleton and an error message.

21 You can lose a half-filled assessment by clicking away

Where Every form in the app.

Problem There is no "you have unsaved changes" warning anywhere, and no autosave. Fill in half a RULA, REBA or RAMP, click away or close the tab, and **everything since your last Save is gone, silently.**

Note These are long multi-page forms. This is probably the bug your users hit most often.

22 The AI report section fails silently

Where Generate report → AI report.

Problem Your organisation has AI analysis **disabled**. Tick "AI report" anyway and it's quietly ignored — no warning. If the AI call fails, that's also swallowed. Either way you get a success message and a document missing the section you asked for.

23 Work Movements numbers differ between report and screen

Where Word report vs measurement screen.

Problem The screen shows an **average**. The report rebuilds a **median** from the chart bars. Same concept, two numbers.

The report's column header is literally "**Mean / Median Velocity**", and it switches per body part — so **one row can be a median and the row below it an average**, under the same heading.

24 The dashboard counters count the wrong people

Where Home, licence bar.

Problem

- **"Active workers this month"** counts people who *logged into the web app*, of any role. Workers don't log in — they're never in that list.
- **"Measurements this month"** includes failed and in-progress ones, double-counting with the "Failed / interrupted" card.
- **"Risk assessments this month"** is counted by who created it, not which organisation it belongs to — so a consultant working in two organisations leaks one into the other.
- **Licence "Workers"** counts admins against the worker limit. "Admins 4/5" doesn't match the 5 extra admins visible on the Team page.

25 Swedish translated exactly one word

Where Language switcher.

Problem Switching to Swedish changed "Home" to "Hem". That's it. Menus, page titles, tabs, column headers, buttons and status labels all stayed English.

The language choice **isn't even saved** — it's deliberately left out of the settings save.

Note You're a Swedish company selling to Swedish customers. This one matters commercially.

26 Three notification settings do nothing

Where Settings → notifications.

Problem "Notify on measurement complete", "on measurement failed", "on risk assessment generated" — all three can be toggled and saved, and are read by **nothing**. They have no effect whatsoever.

CODE EVIDENCE

- The three fields exist on the user (`users/models/baseUserModel.py:166-168`) and are editable through the API (`users/serializers/userSerializer.py:157-159`).
- Notifications are created in exactly **seven** places — 2 in `user_sessions/tasks.py` , 3 in `mec_calculator.py` , 2 in `ramp_calculator.py` . **None of the seven reads any of the three fields**. The notification is sent regardless.

27 Work Library can destroy data quietly

Where Work Library.

- Problem
- **CSV import isn't all-or-nothing**. If row 500 fails, rows 1–499 are already saved. And the useful error from the server ("Missing field 'Factory' in row 12") is thrown away and replaced with "Something went wrong" — so the user can't fix their file.
 - **Deleting anything in the tree deletes measurement tags**. The confirmation says "everything inside it will be permanently removed" — the user reads that as "the folder", not "my measurement data".
 - **Merging two items has no confirmation at all**, and it can't be undone.

Minor

#	Where	Problem
28	RAMP results	Sub-rows are numbered meaninglessly. Inside item 1.4 the rows read "1.1 / 1.2 / 1.3" — but they aren't sub-questions, they're the three variables (forward, side, backward bending), and every item restarts at 1.1. Each row also repeats all three explanation sentences instead of its own. Label them by variable name.
29	RAMP results	"Total limb time: 00:06:36 " on assessment 2224 — but the measurement itself only lasted 06:26 . Ten seconds that don't exist.
30	Risk assessments, Team	No multi-select delete. Five duplicate RAMPs means deleting them one at a time. (Measurements has it and it works properly.)
31	Workers	Workers can't be deleted or deactivated. Someone who leaves stays in the list and the counters forever. Not allowing hard delete is sensible — offering no alternative isn't.
32	Workers	No duplicate check and no validation. You can create fifty "John Smith"s. Age and weight are free text with no limits — age "333" is in your data right now , and "abc" would be accepted. Editing a worker can blank their name entirely.
33	Workers	Search can't find a full name. "John Smith" returns nothing — it only matches the start of the first name <i>or</i> the last name, not both.
34	Measurements	Compare / Merge / Combined report are greyed out with no explanation. The user has no idea they need to select 2–4 measurements.
35	Measurements	A zero-second measurement is treated as valid. #7004 lasts <code>00:00</code> , is marked Completed, and Home offers to run a risk assessment on it.
36	Comparison	Columns say "Session 1 / Session 2" instead of the measurement numbers. Body parts that weren't recorded show as full tables of dashes instead of being hidden.
37	Measurement detail	Shows one "View RAMP" link even though five exist. Duplicates are invisible from there.
38	RULA / REBA	They lost their PDF export in the new version. The old one worked.
39	RULA / REBA	Results show invented "Action needed" badges — any sub-score of 3 or more gets one. That's not part of either method. It labels a raw input (like coupling or force) as a risk verdict. The final scores are correct; these badges are made up and could mislead a customer.
40	Work Library	The drag handle on tasks is fake — the grip icon is drawn, but there's no drag code and no ordering field in the database. Tasks are always shown oldest-first.
41	Work Library	"N measurements" is always 0, so the "low use" warning can never appear. The count is wired correctly — but nothing ever fills the table it reads from.
42	Work Library	The CSV import box claims a 5 MB limit that isn't enforced anywhere, and accepts .xlsx despite the button saying "Import CSV".
43	Settings	Help & Support: all three cards are dead. "Create a support ticket", "Documentation", "Email support" look clickable and do nothing. The version number "v2.4.0" is typed in by hand.
44	Run assessment	"Onset (T1°)" is labelled in degrees. It's actually seconds . The user is told 5 means 5 degrees; the backend treats it as 5 seconds.

45	Measurement detail	Hand distance from the body is an <i>estimate</i> presented as a measurement. It assumes a 32 cm upper arm and 27 cm forearm for every worker, regardless of their size.
46	Public site	The <code>/prototype</code> pages are reachable with no login , on the live domain. They don't touch real data, but anyone can open them.
47	Everywhere	Plurals are hardcoded: "1 lines", "1 workstations", "1 activities". Gender shows the raw value inconsistently — "male" next to "MALE" in the same column. Times under one second show as <code>00:00:00</code> next to a non-zero percentage. The heart-rate table shows "RMSDD" — a typo of RMSSD.
48	Every page	Errors in the browser console on every load (Firebase App Check, plus an unhandled Sentry rejection). Harmless today, but they hide real errors.

Data that should be deleted

Test Hub has 20 assessments. **Fourteen are broken.** Several assessments on one measurement is normal — none are listed here for that reason.

Group	IDs	What's wrong
A — Empty (9)	2180, 2181, 2182, 2183, 2184, 2185, 2188, 2189, 2190	Stuck at "In progress" with nothing inside . Generation ran and produced no result. These are what make measurement #7010 look like it has five assessments.
B — All zeros (2)	2177, 2178	Marked Completed, but every number is 0 and two questions are empty. 2177 isn't even linked to a measurement.
C — Old versions (3)	2042, 2044, 2045	The ones you said you already ignore. Old format, not linked to any measurement.
Leave alone	2223, 2224	Real, valid RAMPs — just identical duplicates of each other. That's bug #6, not corruption. Deleting one is your call, and it will happen again until #6 is fixed.

Not tested

- **Importing a measurement ZIP file.** I checked enough to know the upload accepts **any file, of any size, of any type** — no checks at all — and that every error becomes a generic "server error". But I did not test what a malicious or corrupt file does. **This is the one place with a real security risk. It should be looked at next.**
- Login, logout, password reset, accepting an invitation.
- Phone and tablet layouts.
- Filling in a RAMP's manual questions from start to finish.